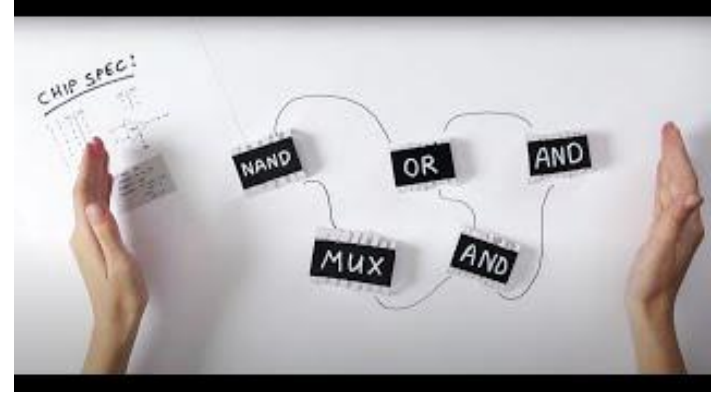




CSc 360

Operating Systems

Processes

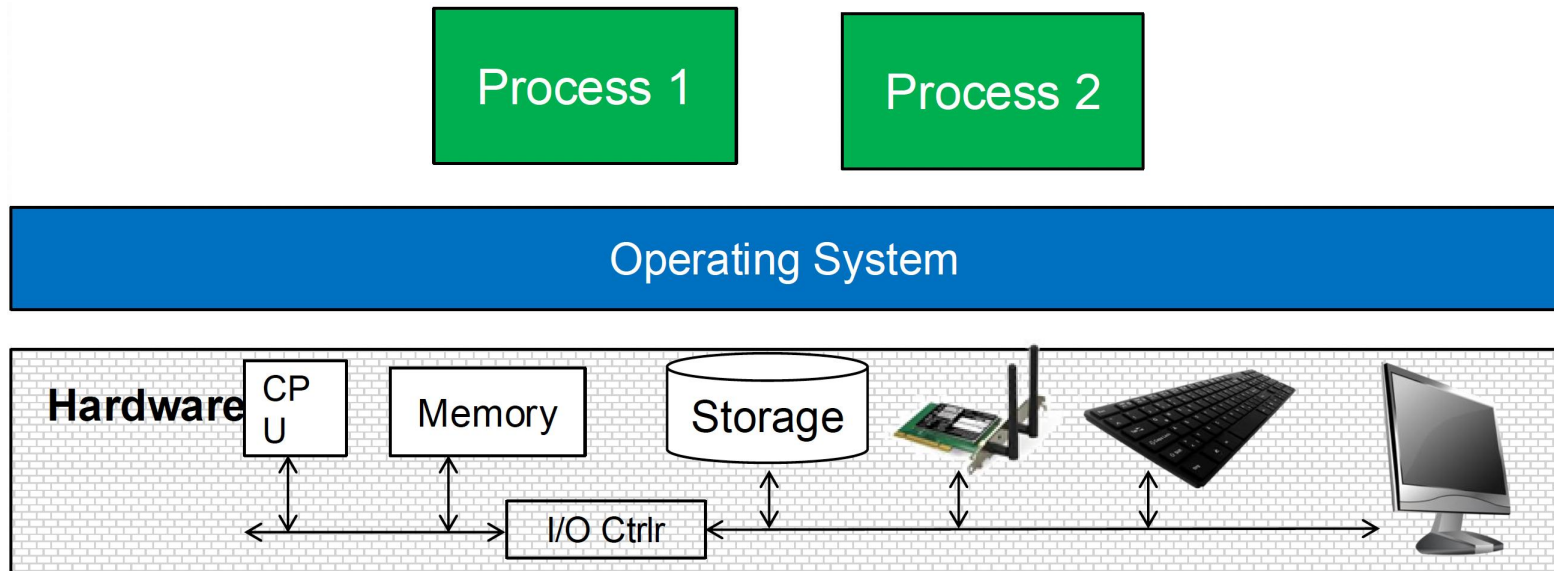
Wenjun Yang

Summer 2026

Recall: Topic Breakdown (Course Objective)

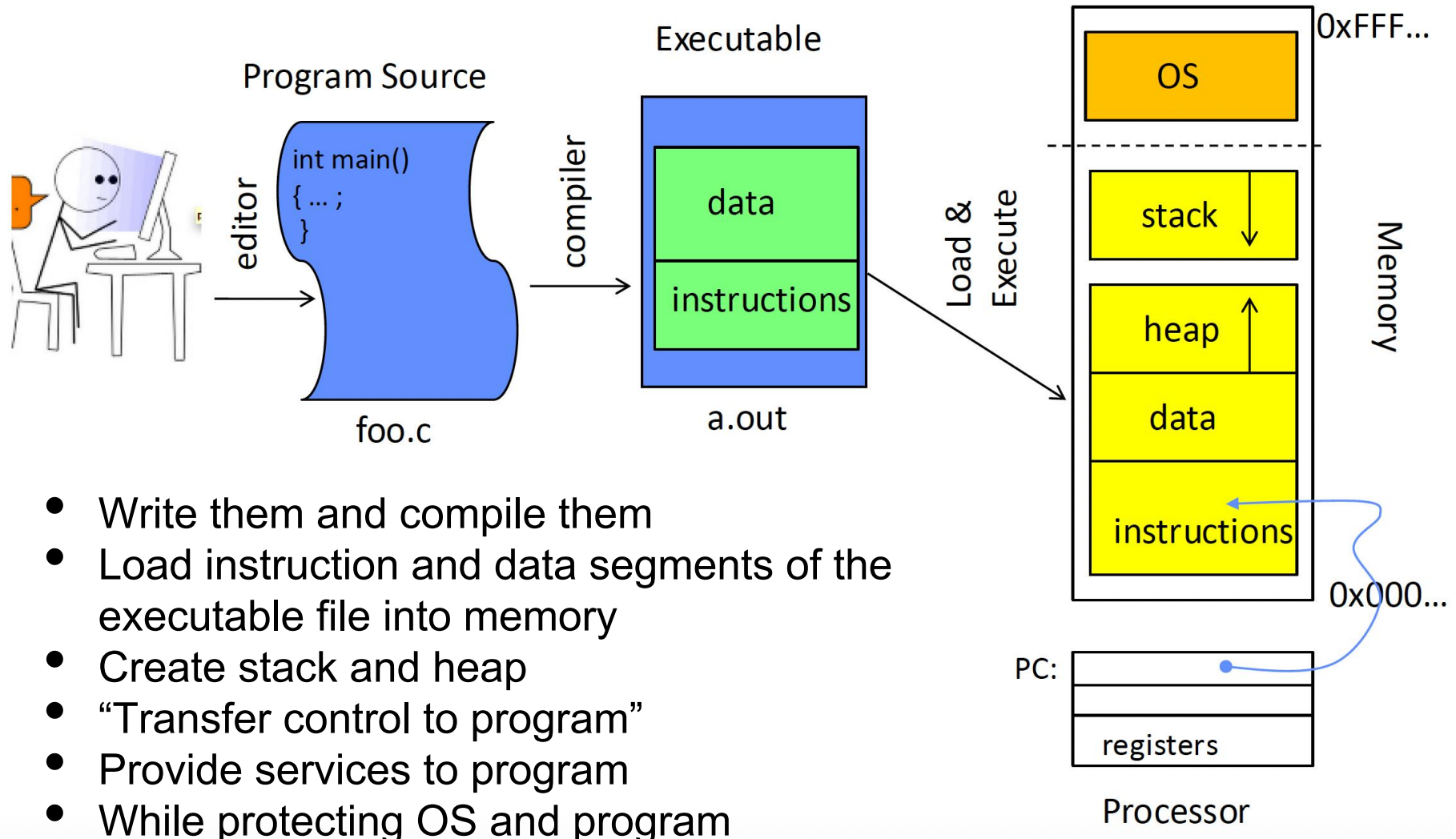
- virtualizing the CPU (**process**):
process, thread, scheduling, synch
- virtualizing memory (**memory**):
memory management, virtual memory
- **storage**: file systems, I/O systems

Process: an active instance of a program



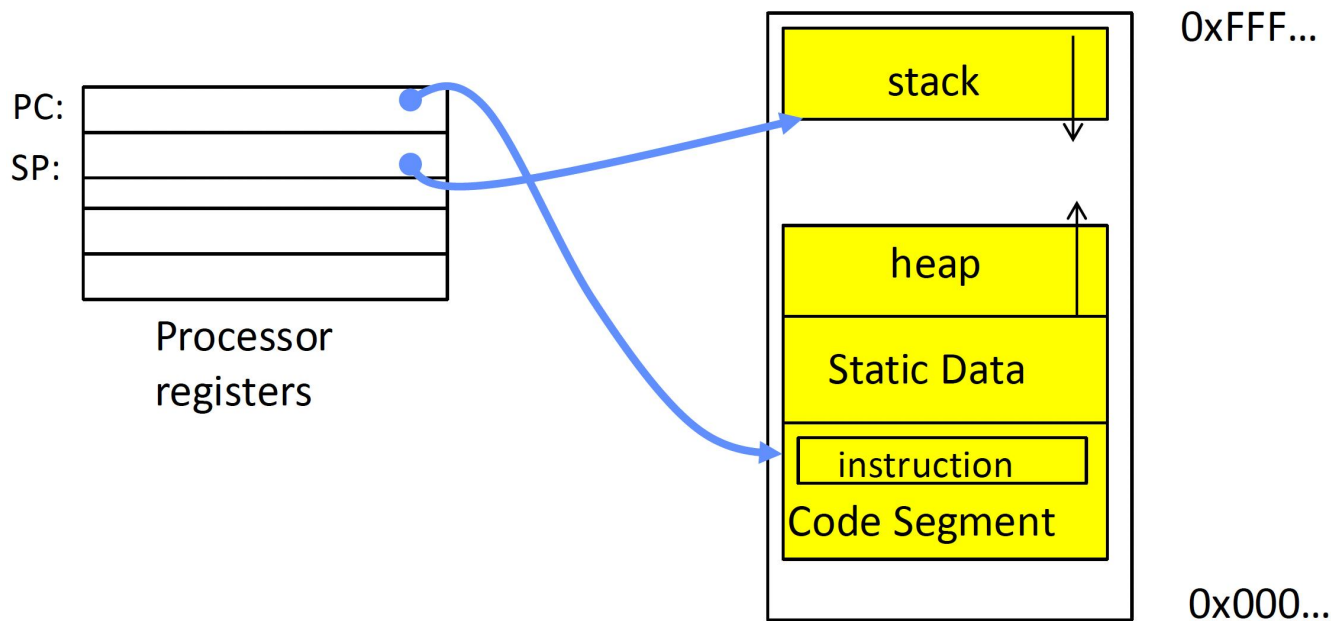
- OS virtualizes hardware to processes (applications) to provide the illusion that each process uses its own machine; also provides illusion of infinite memory and processor

OS Bottom Line: Run Programs



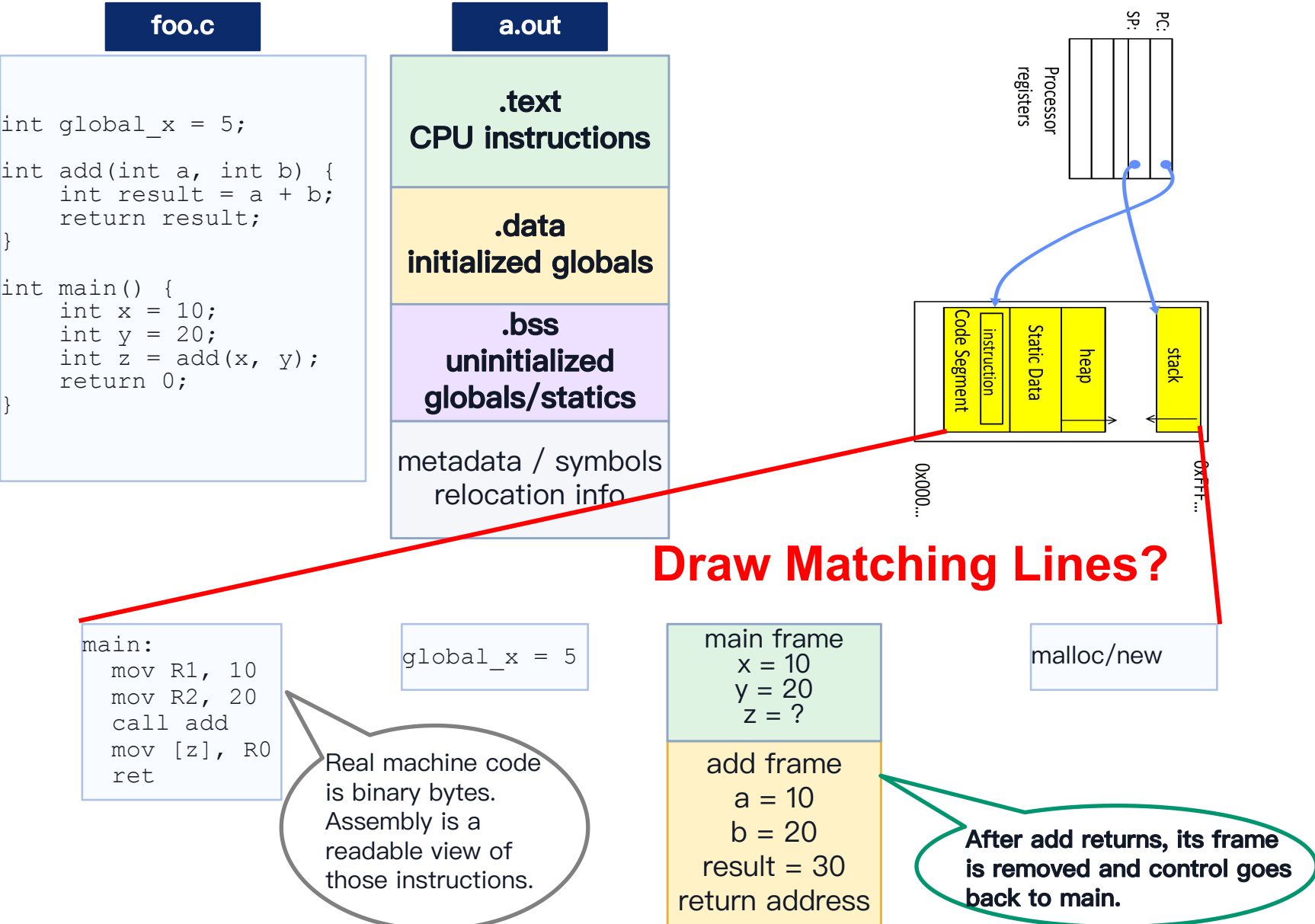
- Write them and compile them
- Load instruction and data segments of the executable file into memory
- Create stack and heap
- “Transfer control to program”
- Provide services to program
- While protecting OS and program

Address Space



- What's in the code segment? Static data segment?
- What's in the stack segment?
 - How is it allocated? How big is it?
- What's in the heap segment?
 - How is it allocated? How big?

OS Creates a Process

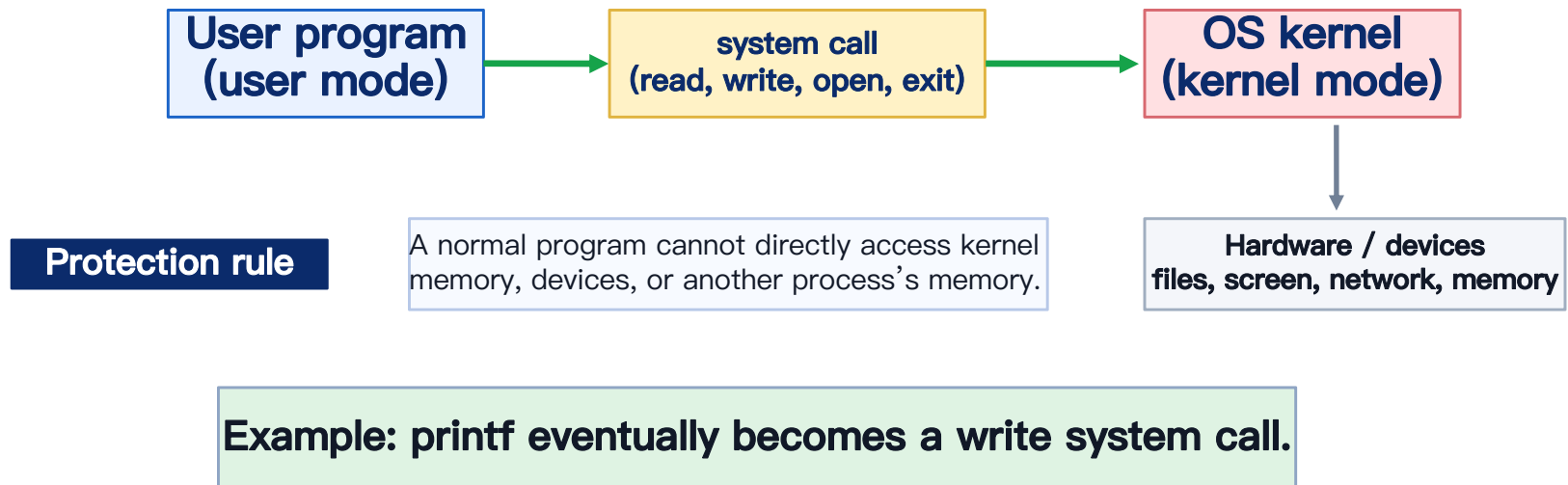


OS Jobs

- **Create Process**

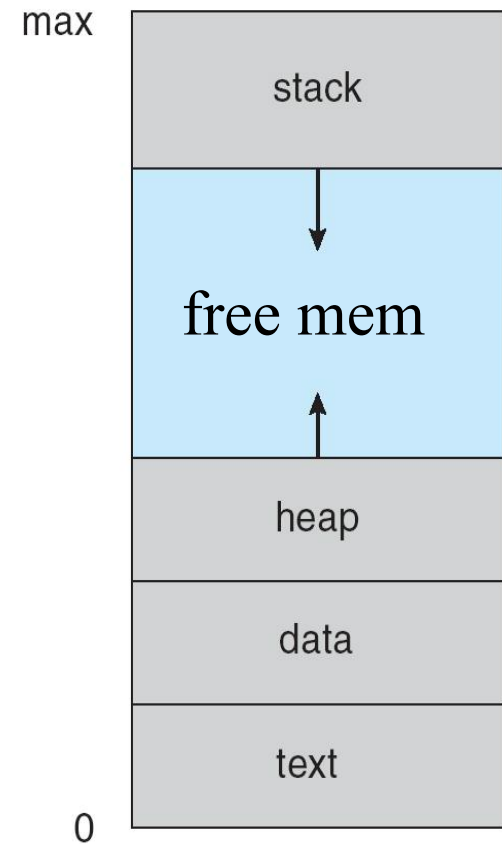
- Load .text, .data, .bss into memory
- Create stack and heap
- Initialize registers
- Set PC to entry point
- Transfer control to user code

- **Privileged kernel services**



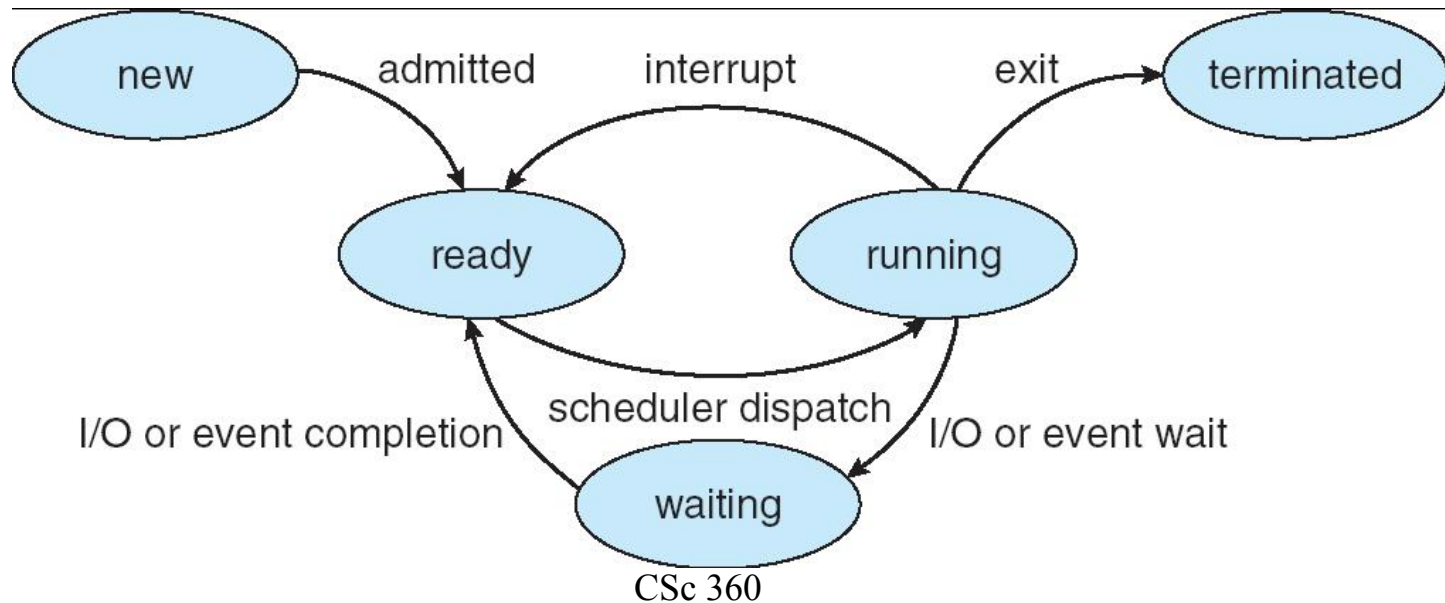
Processes

- Process: a program in execution
- Program: passive entity
 - static binary file on storage
 - e.g., `gcc -o hello hello.c`
- Process: active entity; resource allocated!
 - `./hello`
 - text (code); data (static), stack, heap
 - process control block (PCB)



Process states

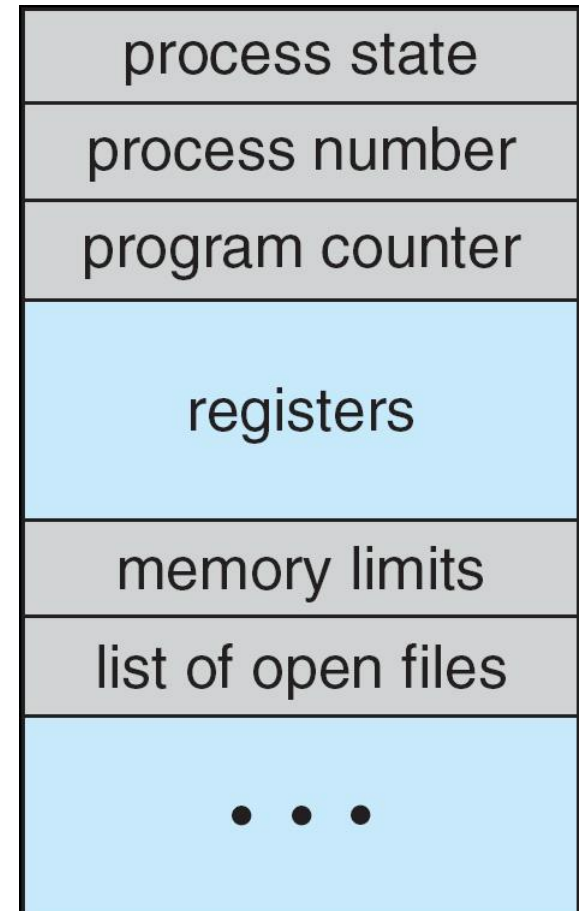
- E.g., one CPU (core)
 - one running process at any time
 - maybe many ready/waiting processes



why interrupt?

Process control blocks

- PCB: keep track processes
 - state: ready/running, etc
 - CPU
 - PC, registers, priority, etc
 - memory
 - memory control information
 - I/O
 - e.g., list of opened files
 - accounting

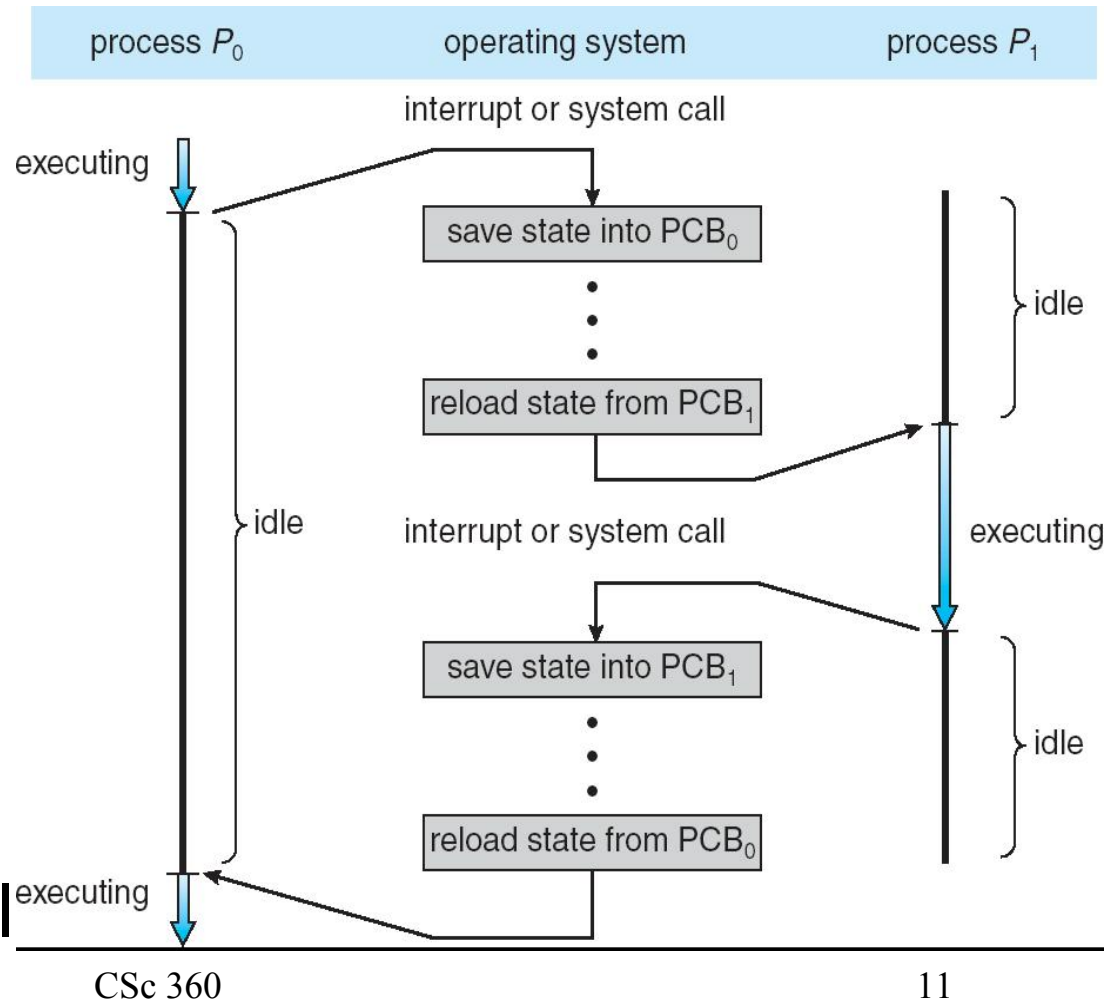


Context switching

- Context switch
 - save states
 - restore states

- When

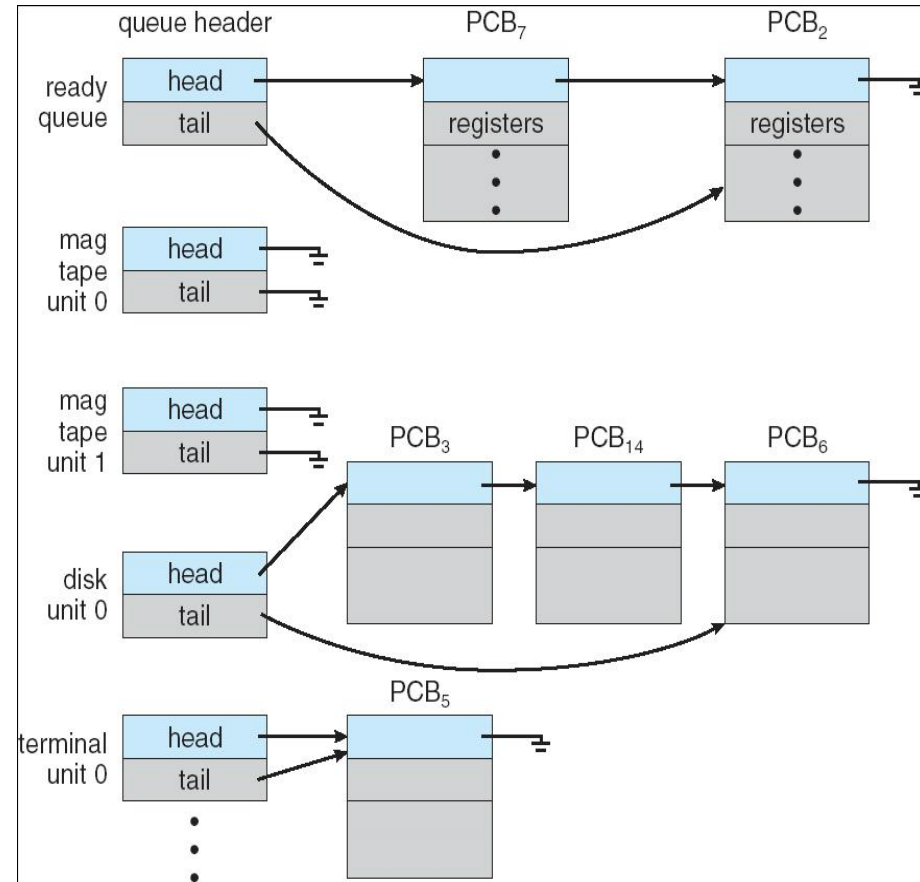
- timer
- I/O, memory
- trap
- waiting sys call



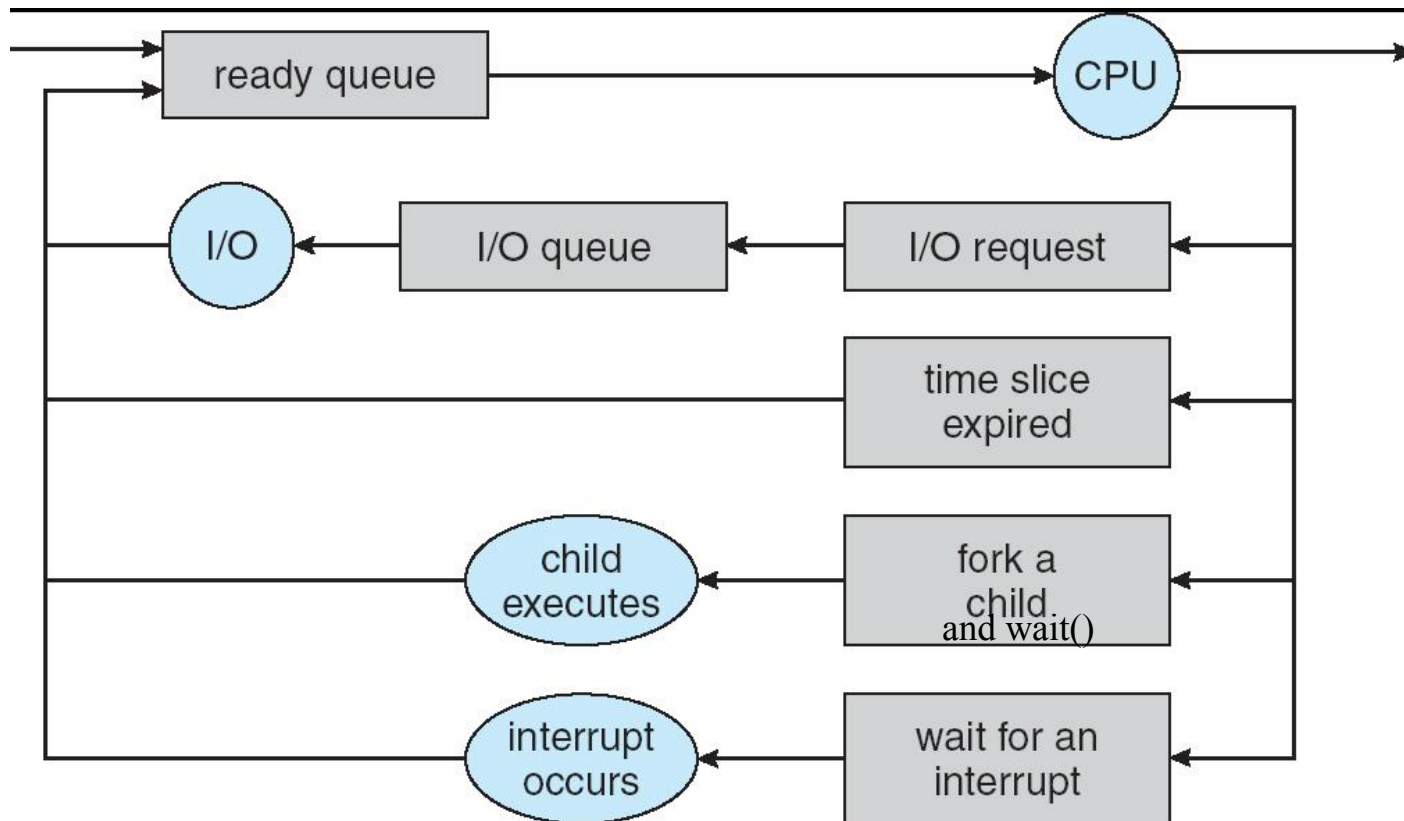
when more than two processes

Process scheduling

- Multiprogramming
 - utilization
- Timesharing
 - interactive
- Scheduling queues
 - linked list
 - ready queue
 - I/O queue



Queuing system



Queuing scheduler

- Who's the next?
- Long-term scheduler
 - job scheduler (spooling)
 - get to the ready queue
 - CPU-intensive vs I/O intensive
- Short-term scheduler
 - CPU scheduler
 - frequency vs overhead

More on scheduling

- Medium-term scheduler
 - who is NOT the next
 - reduce the degree of multiprogramming
 - swap-in/out
- Scheduling algorithms
 - first-come-first-server, shortest-job-first, priority, round-robin, fair and weighted fair, ...
 - more in Chapter 5

This lecture so far

- Process and process scheduling
 - process vs program
 - process control block
 - context switch: what to save/restore
 - process scheduling
- Explore further
 - process status: `/bin/ps`
 - top CPU processes: `/usr/bin/top`